Programming Assignment Report

Design and Analysis of Algorithms

Name: Chunduru Thanusha

Registration Number: 2411021060649

Problem Name: Insertion Sort – Insert Element into Sorted List

1. Problem Statement

Given a sorted array with one unsorted element at the end, insert that element into its correct position so that the array remains sorted. During the process, print the array after each shift or insertion step.

2. HackerRank Problem Link

https://www.hackerrank.com/challenges/insertionsort1

3. Github Submission Link

https://github.com/Thanushadaa/DAA-Hacker-Rank-Assignment/tree/main

4. Algorithm

- Start the program.
- Input integer n (size of array).
- Input array arr of size n.
- Store the last element as key.
- Initialize i = n - 2.
- Compare key with elements from right to left.
- Shift elements greater than key to the right and print array.
- Insert key at correct position.
- Print final array.
- Stop the program.

5. Example Explanation

Input: n = 5, arr = [2, 4, 6, 8, 3]

Steps:

1. [2, 4, 6, 8, 8]
2. [2, 4, 6, 6, 8]
3. [2, 4, 4, 6, 8]
4. [2, 3, 4, 6, 8]

6. Complete Java Code

```
import java.util.*;

public class Solution {

  public static void insertionSort(int n, int[] arr) {
    int key = arr[n - 1];
    int i = n - 2;

    while (i >= 0 && arr[i] > key) {
      arr[i + 1] = arr[i];
      printArray(arr);
      i--;
    }

    arr[i + 1] = key;
    printArray(arr);
  }

  public static void printArray(int[] arr) {
    for (int num : arr) {
      System.out.print(num + " ");
    }
    System.out.println();
  }

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int[] arr = new int[n];

    for (int i = 0; i < n; i++) {
```

```
            arr[i] = sc.nextInt();
        }

        insertionSort(n, arr);
        sc.close();
    }
}
```

7. Observation / Solution

- The last element is taken as the key.
- Elements greater than the key are shifted right.
- The key is inserted at the correct position.
- The array is printed after each step.

8. Time Complexity

Worst Case: O(n)

Best Case: O(1)